

Introduction to Message Passing Interface (MPI)

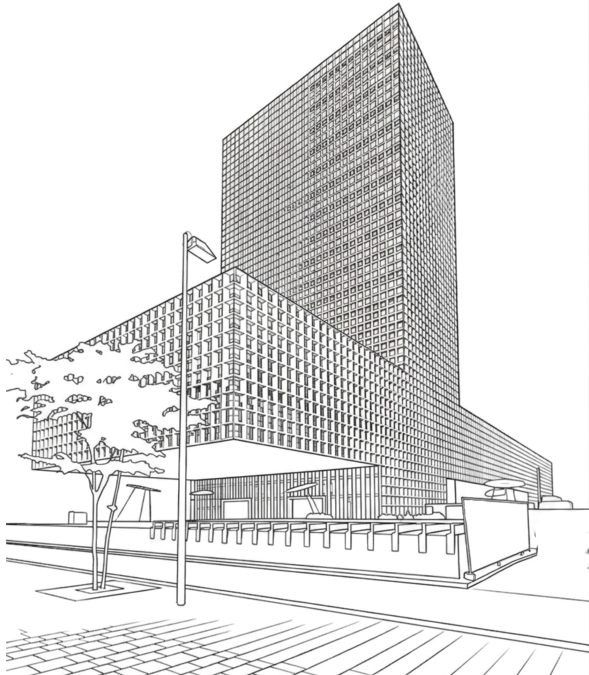
Lesson 3

Matteo Barborini

University of Luxembourg - HPC Platform

matteo.barborini@uni.lu

30th of September 2026



Solution: Basic numerical Integration



Use as an example the serial code `1.3.SERIAL_TrapezodialRule_sin.c`, update the function to integrate and the integration interval and parallelize the code using `MPI_send` and `MPI_recv`.

Integrate the function cosine in the interval $[-\pi/2, \pi/2]$.

1. Rank 0 divides the integral space $[-\pi/2, \pi/2]$ into $\mathcal{M}$ elements equal to the total number of MPI processes and computes the interval h .
2. Rank 0 communicated the value of h to all MPI processes
3. Each process (even rank 0) integrates over their subspace $[c_m, c_{m+1}]$ obtaining I_m values.
4. Collect all values from each process to the main one, (rank 0)
5. Rank 0 sums the collected values and prints the integral.

We define the function that computes the integral in a finite interval

```
1 double integral_of_my_function(double *a, double *b, int *num_of_intervals)
2 {
3     // Invert order of a and b if a > b
4     if (*a > *b)
5     {
6         double c = *b;
7         *b = *a;
8         *a = c;
9     }
10    double integral = 0.0;
11    if (*num_of_intervals > 0)
12    {
13        // Compute the width of each trapezoid
14        double h = (*b - *a) / ((double)(*num_of_intervals));
15        // Compute Integral
16        integral = my_function(a) / 2.0;
17        for (int i = 1; i < *num_of_intervals; i++)
18        {
19            double x = *a + h * (double)(i);
20            integral += my_function(&x);
21        }
22        integral += my_function(b) / 2.0;
23        integral = h * integral;
24    }
25    return integral;
26 }
```

Solution: 1.MPI_TrapezoidalRule_cos.c

Solution: Basic numerical Integration

The initialization of the variables after the MPI environment is built (all tasks have this information)

```
1  MPI_Init(&argc, &argv);
2  int num_of_ranks;
3  MPI_Comm_size(MPI_COMM_WORLD, &num_of_ranks);
4  int mpi_rank;
5  MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
6
7  int num_of_intervals = 10008;
8
9  // Integration limits for cos(x) from -pi/2 to pi/2
10 double a = -0.5 * M_PI;
11 double b = 0.5 * M_PI;
12
13 // Invert order of a and b if a > b
14 if (a > b)
15 {
16     double c = b;
17     b = a;
18     a = c;
19 }
20
21 // Compute the width of each trapezoid
22 double h = (b - a) / ((double)(num_of_intervals));
```

Next we compute intervals for each MPI task

```
1  int num_of_intervals_per_rank = (int)num_of_intervals / num_of_ranks;  
2  int num_of_residual_points = (int)num_of_intervals % num_of_ranks;  
3  
4  // Compute integration points and intervals per rank.  
5  if (mpi_rank == 0)  
6  {  
7      num_of_intervals_per_rank += num_of_residual_points;  
8      b = a + h * num_of_intervals_per_rank;  
9  }  
10 else  
11 {  
12     a = a + h * (mpi_rank * num_of_intervals_per_rank + num_of_residual_points);  
13     b = a + h * num_of_intervals_per_rank;  
14 }
```

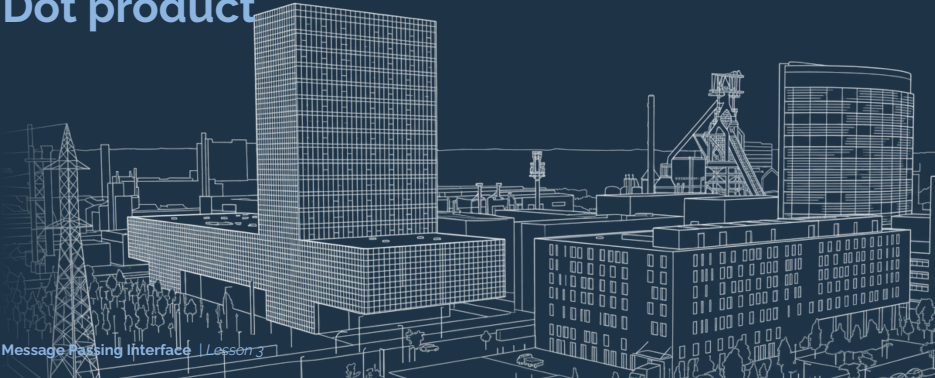
Notice that if `num_of_intervals_per_rank = 0` then some ranks will have `a = b`.

The function that computes the integral will check that the number of points is greater than zero, otherwise it will return 0.0.

Every MPI rank computes the integral, for some ranks the result will be 0.0

```
1 // Compute local integrals
2 double integral = integral_of_my_function(&a, &b, &num_of_intervals_per_rank);
3
4 // Communicate the results of the integrations to rank 0 for final summation
5 if (mpi_rank != 0)
6 {
7     MPI_Send(&integral, 1, MPI_DOUBLE, 0, mpi_rank, MPI_COMM_WORLD);
8 }
9 else
10 {
11     double integral_tmp = 0.0;
12     for (int i = 1; i < num_of_ranks; i++)
13     {
14         MPI_Recv(&integral_tmp, 1, MPI_DOUBLE, i, i, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
15         integral += integral_tmp;
16     }
17     // Print result
18     printf("Approximate Integral: %.10E\n", integral);
19 }
```

Solution: Dot product



Consider two vectors **a** and **b** of N elements each, compute their dot product defined as

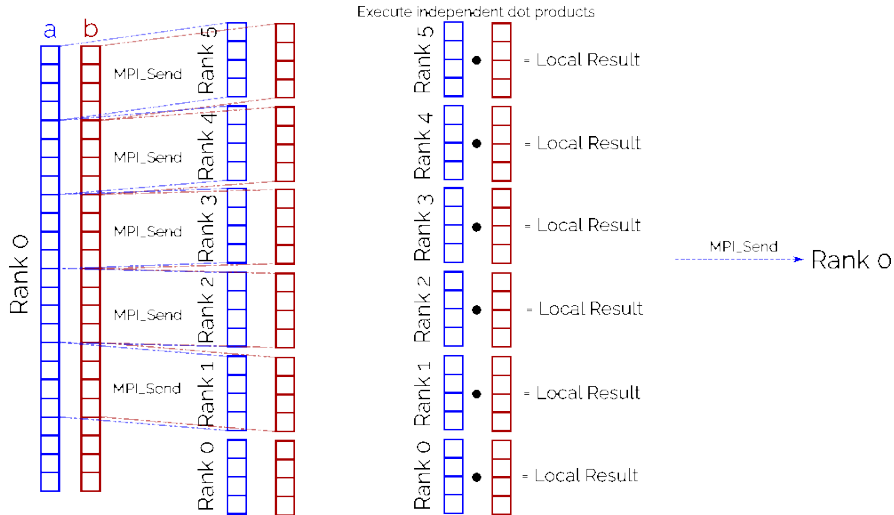
$$c = \mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^N a_i b_i.$$

Parallelize the operation with MPI_Send and MPI_Recv.

1. Rank 0 divides the vectors in M segments equal to the total number of MPI processes. (Warning!! One rank might have more or less elements)
2. Rank 0 communicates the value of the lengths of each segment to the other ranks.
3. Rank 0 communicates the segments to each corresponding rank.
4. Each process multiplies the partial vectors. They do separate dot products.
5. Collect all values from each process to the main one, (rank 0)
6. Rank 0 sums the collected values and prints the integral.

Definition of dot product function

Solution: Dot product



We first define the function to compute the dot product

```
1 double ddot(double *a, double *b, int *num_of_elements)
2 {
3     double c = 0.0;
4     if (*num_of_elements > 0)
5     {
6         for (int i = 0; i < *num_of_elements; i++)
7         {
8             c += a[i] * b[i];
9         }
10    }
11    return c;
12 }
```

following a similar nomenclature used in the BLAS libraries. Here the first **ddot** refers to double precision arrays.

This function can be executed by one or multiple processes at the same time.

After the initialization of the MPI environment the application we define the variables for **ALL RANKS**

```
1 int num_of_elements = 100;
2 double *a;
3 double *b;
```

Afterwards rank 0 allocates the **a** and **b** vectors

```
1  if (mpi_rank == 0)
2  {
3      a = (double *)calloc(num_of_elements, sizeof(double));
4      b = (double *)calloc(num_of_elements, sizeof(double));
5      for (int i = 0; i < num_of_elements; i++)
6      {
7          a[i] = (double)i;
8          b[i] = (double)i;
9      }
10 }
```

Notice that both vectors represent consecutive blocks of data.

All ranks compute the number of elements on which they have to compute the dot product

```
1  int num_of_elements_per_rank = (int)num_of_elements / num_of_ranks;
2  int num_of_residual_elements = (int)num_of_elements % num_of_ranks;
```

The residual elements are added to **Rank 0**.

```
1  if (num_of_elements_per_rank > 0)
2  {
3      if (mpi_rank == 0)
4      {
5          int first_element = 0;
6          for (int i = 1; i < num_of_ranks; i++)
7          {
8              first_element = i * num_of_elements_per_rank + num_of_residual_elements;
9              MPI_Send(&a[first_element], num_of_elements_per_rank, MPI_DOUBLE,
10                     i, i, MPI_COMM_WORLD);
11              MPI_Send(&b[first_element], num_of_elements_per_rank, MPI_DOUBLE,
12                     i, i + num_of_ranks, MPI_COMM_WORLD);
13          }
14      }
15      else
16      {
17          a = (double *)calloc(num_of_elements_per_rank, sizeof(double));
18          b = (double *)calloc(num_of_elements_per_rank, sizeof(double));
19          MPI_Recv(&a[0], num_of_elements_per_rank, MPI_DOUBLE, 0, mpi_rank,
20                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
21          MPI_Recv(&b[0], num_of_elements_per_rank, MPI_DOUBLE, 0, mpi_rank + num_of_ranks,
22                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
23      }
24  }
```

Solution: 2.MPI_DotProduct.c

Solution: Dot product

```
1 // Add the residual elements to the number of elements of rank 0
2 if (mpi_rank == 0)
3 {
4     num_of_elements_per_rank += num_of_residual_elements;
5 }
6
7 // Each Rank computes the dot product if it has points to compute
8 double result = ddot(&a[0], &b[0], &num_of_elements_per_rank);
9
10 // Communicate the results of the integrations to rank 0 for final summation
11 if (mpi_rank != 0)
12 {
13     MPI_Send(&result, 1, MPI_DOUBLE, 0, mpi_rank, MPI_COMM_WORLD);
14 }
15 else
16 {
17     double result_tmp = 0.0;
18     for (int i = 1; i < num_of_ranks; i++)
19     {
20         MPI_Recv(&result_tmp, 1, MPI_DOUBLE, i, i, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
21         result += result_tmp;
22     }
23
24     // Print result
25     printf("Value of the dot product: %.10E\n", result);
26 }
```

Performances and scaling



We achieve **linear speedup** if given an initial task that takes time T_1 to complete on a single processor, the same task can be completed in time $T_p = T_1/p$ on p processors.

T_1 Time in which the application runs with one process;

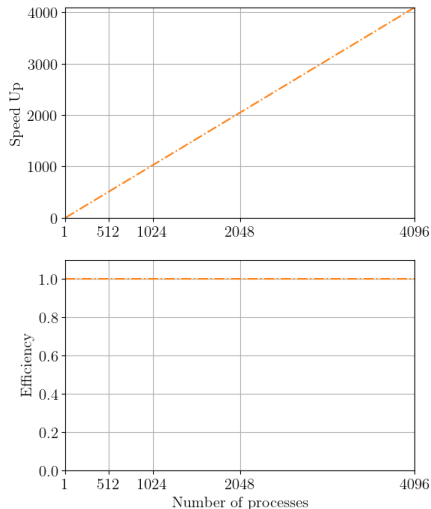
p Number of parallel processes;

T_p Time in which the application runs with p processes.

We define the **speedup** S and **efficiency** E of the parallel program as

$$S = \frac{T_1}{T_p}, \quad E = \frac{S}{p} = \frac{T_1}{pT_p}$$

So linear speedup has $S = p$ and $E = 1$.



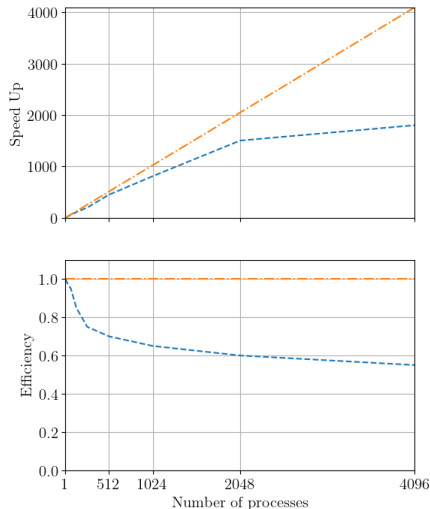
In practice, we will almost always have $S < p$ and $E < 1$.

- If we **fix the problem size**, S will usually be lower than the linear function as we increase p ;
- If we **the problem size increase proportionally to the number of processes/threads**, then E often decreases below the ideal constant.

This is because we also need to consider the '**parallel overhead**' time T_o that is the time a system requires to handle concurrent tasks or data communication.

$$T_p = T_1/p + T_o$$

T_o Overhead time for handling concurrent tasks or data communication, and includes the non parallelizable part of the code



Amdahl's law¹, is "pessimistic" in the sense that it assumes a **fixed problem size**, states that the maximum speedup of a program using multiple processors is limited by the sequential fraction of the program.

If f_s is the fraction of the program that must be executed sequentially, the parallel time will be

$$T_p = (1 - f_s) \frac{T_1}{p} + f_s T_1$$

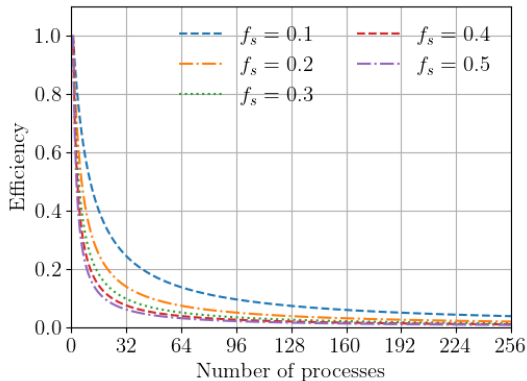
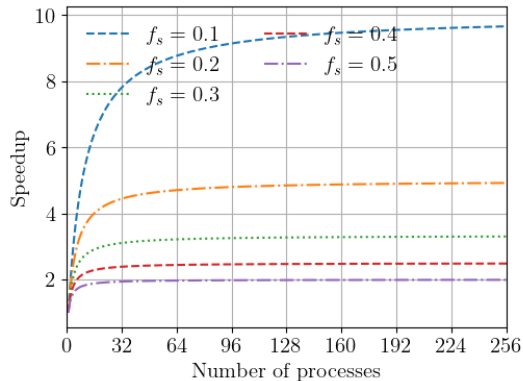
and then the speedup S achievable with p processors is given by:

$$S_{max} \geq S = \frac{T_1}{T_p} = \frac{1}{f_s + \frac{1-f_s}{p}}$$

As $p \rightarrow \infty$ the maximum speedup approaches $S \rightarrow 1/f_s$.

If the serial part of the code still amounts to 10% of the remaining computational time, the maximum speedup is 10, even for $p \rightarrow \infty$.

¹G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities", in Afips '67 (spring) (1967).



Gustafson–Barsis's law² is “optimistic”: **it is possible to increase the size of the problem with the number of processes available, thus increasing efficiency.**

If we assume that the parallel time grows with the system size we have that the time the serial process would take to compute the same task is:

$$T_1 = f_s T_p + (1 - f_s) T_p \cdot p$$

while clearly

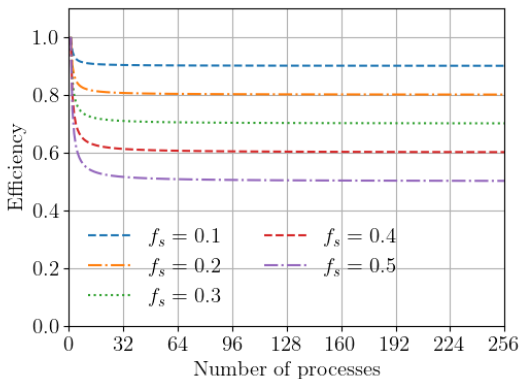
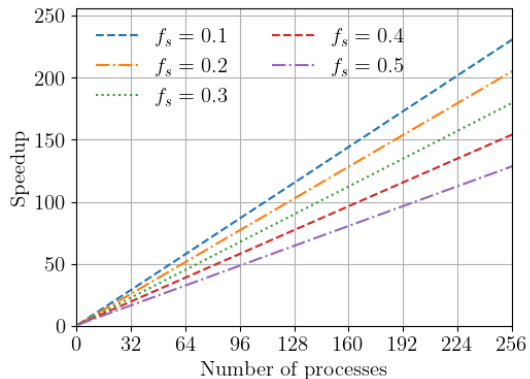
$$T_p = f_s T_p + (1 - f_s) T_p$$

so the speedup is

$$S = \frac{T_1}{T_p} = f_s + (1 - f_s) \cdot p$$

We assume that the serial part of the code does not grow with the problem size.

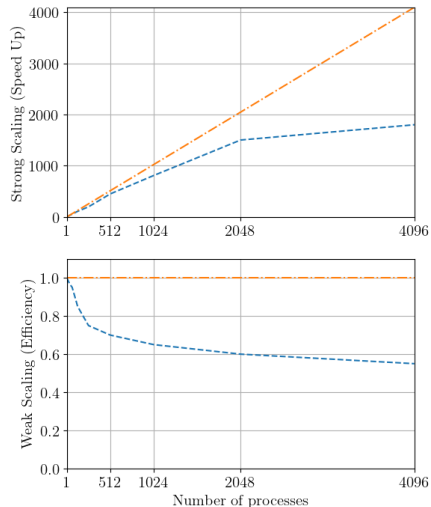
²J. L. Gustafson, “Reevaluating Amdahl’s law”, Communications of the ACM **31**, 532–533 (1988).



Thus the scalability of a program is measured by:

Strong scaling (Speed Up): The problem size is fixed, and the number of processors is increased. (The speedup is expected to be linear with the number of processors)

Weak scaling (Efficiency): The problem size increases proportionally with the number of processors. (The efficiency is constant)

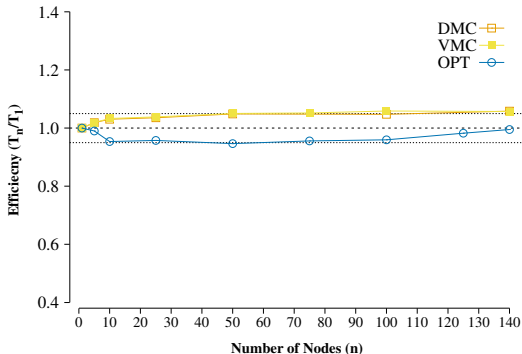


In QMeCha we showed a fixed molecular system, a cluster $(H_2O)_{30}$ of water molecules.

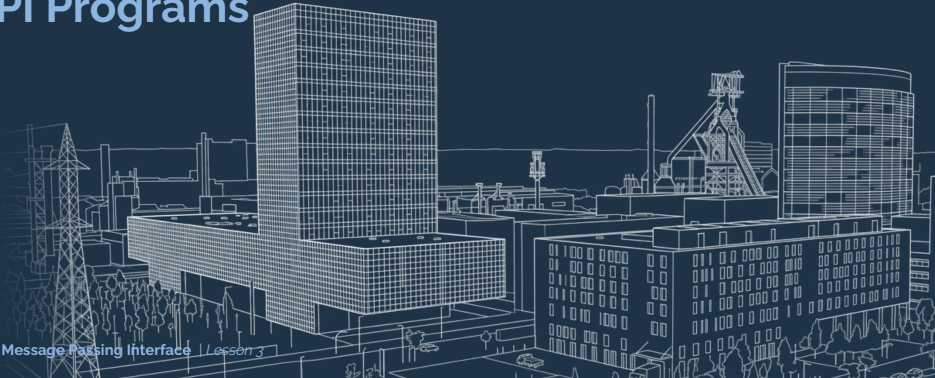
We use a number of independent Monte Carlo integration instances that are proportional to $\#Nodes \times \#MPI_tasks_per_node \times \#OpenMP_threads$ number of processes.

Increasing the number of independent samplings corresponds to the increase of the problem.

We have nearly perfect weak scaling.



Timing MPI Programs



Returns a double-precision number of seconds, usually the system time.

```
1 double MPI_Wtime();
```

Returns as a double-precision value, the number of seconds between successive clock ticks.

```
1 double MPI_Wtick();
```

So within the example `3.1.MPI_Wtime_Wtick.c` the commands are simply used by calling them **after** the initialization of the MPI environment

```
1 accuracy_time = MPI_Wtick();
2 if (mpi_rank == 0) {
3     printf("MPI timings are computed with an accuracy of : %10.3e seconds\n", accuracy_time
4     );
5 }
```

and

```
1 starting_time = MPI_Wtime();
2
3 // ... do all operations ...
4
5 ending_time = MPI_Wtime();
6 printf("MPI rank %d, Time taken to send and receive vector : %f seconds\n", mpi_rank,
7     ending_time - starting_time);
8
9 // Finalize the MPI environment
10 MPI_Finalize();
```

Preprocessor for MPI wrapping



```
1  #if defined _MPI
2  #include <mpi.h>
3  #endif
4  #include <stdio.h>
5
6  int main(int argc, char *argv[]){
7
8      int num_of_ranks;
9      int mpi_rank;
10
11     #if defined _MPI
12         MPI_Init(&argc, &argv);
13         MPI_Comm_size(MPI_COMM_WORLD, &num_of_ranks);
14         MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
15     #else
16         num_of_ranks = 1;
17         mpi_rank = 0;
18     #endif
19
20     printf("Hello world from MPI process %d out of %d processes\n",
21           mpi_rank, num_of_ranks);
22
23     #if defined _MPI
24         MPI_Finalize();
25     #endif
26     return 0;
27 }
```

Through the use of the preprocessor in C we can introduce a variable that is used automatically to switch of the MPI calls, in order to make the serial version of a code compilable without MPI libraries (if not present).

```
#if defined _MPI
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD, &num_of_ranks);
    MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
#else
    num_of_ranks = 1;
    mpi_rank = 0 ;
#endif
```

In this case `_MPI` is the variable that is checked. In order to compile the code with mpi we thus need to write:
While to compile the serial version that does not need MPI we can simply do.

In this case `_MPI` is the variable that is checked. In order to compile the code with mpi we thus need to write:

```
mpicc -o myapplication.exe myapplication.c -D_MPI
```

While to compile the serial version that does not need MPI we can simply do.

```
gcc -o myapplication.exe myapplication.c
```

Trying to compile the serial version with the MPI flag will lead to a compilation error since you will try to load the OpenMPI libraries that are not linked.

So

```
gcc -o myapplication.exe myapplication.c -D_MPI
```

FAILS!!

Exercise: Matrix Vector multiplication (dgemv)



We consider a matrix $\mathbf{A}$ with N rows and M columns, and a vector $\mathbf{b}$ of length M , both containing double precision elements.

We define the matrix vector product between the transposed of $\mathbf{A}$ and the vector $\mathbf{b}$ as $\mathbf{c} = \mathbf{A}^T \mathbf{b}$ and the single element of the $\mathbf{c}$ will be given by

$$c_i = \sum_{j=1}^M A_{ij} b_j$$

Exercise: Write a serial code to do a Matrix vector operation, and then parallelize it with point-to-point communications MPI_Send and MPI_Recv.

1. Rank 0 Initializes matrix $\mathbf{A}$ and vector $\mathbf{b}$.
2. Rank 0 communicates to all other ranks the vector $\mathbf{b}$ and parts of the matrix $\mathbf{A}$ necessary for the local matrix vector multiplication.
3. Each process computes the local matrix vector product (Local partial segments of vector $\mathbf{c}$).
4. Each process sends its partial segment of the vector $\mathbf{c}$ to Rank 0.
5. Rank 0 collects all the partial segments of the vector $\mathbf{c}$ from each process.
6. Rank 0 prints vector $\mathbf{c}$.

Examples of matrix allocations in C can be found in the example `4.1.MatrixAllocations.c`

The simplest way to allocate matrices in C is to first allocate a vector of pointers and then for each pointer a vector of length column.

```
1 // Allocate empty matrix
2 double **allocate_2d_double(int rows, int columns)
3 {
4     if (rows <= 0 || columns <= 0)
5         return NULL;
6
7     // Allocate memory for the row pointers (an array of double*)
8     double **matrix = (double **)malloc(rows * sizeof(double *));
9
10    // Allocate memory for the elements of each row (and initialize to 0)
11    for (int i = 0; i < rows; i++)
12    {
13        matrix[i] = (double *)calloc(columns, sizeof(double));
14    }
15    return matrix;
16 }
```

The memory is not necessarily contiguous!! What can happen in MPI transfers of non-contiguous data?

An alternative way to allocate contiguous memory for a matrix is to allocate a single vector of length $\text{rows} \times \text{columns}$.

If we define the function

```
1 // Allocate empty vector
2 double *allocate_1d_double(int elements)
3 {
4     if (elements <= 0)
5         return NULL;
6
7     // Allocate memory for the pointers (an array of double*)
8     double *vector = (double *)calloc(elements, sizeof(double));
9     return vector;
10 }
```

We simply allocate a vector of length $\text{rows} \times \text{columns}$

```
1 vector = allocate_1d_double(rows * columns);
```

The element i, j of the matrix can then be accessed as `vector[i*columns + j]` assuming the allocation is again "columns first".

Remember that in Fortran it is the opposite, Fortran uses "rows first" allocation.

An alternative way to allocate contiguous memory for a matrix is to allocate a single temporary vector of length `rows * columns`.

Afterwards, the matrix can be allocated and the pointers can be associated to the ones used for the larger vector.

```
1 // Allocate empty matrix (consecutive elements)
2 double **allocate_2d_double_blocked(int rows, int columns)
3 {
4     if (rows <= 0 || columns <= 0)
5         return NULL;
6
7     /* allocate the n*m contiguous items */
8     double *temp_matrix = (double *)malloc(rows * columns * sizeof(double));
9
10    /* allocate the row pointers into the memory */
11    double **matrix = (double **)malloc(rows * sizeof(double *));
12
13    /* set up the pointers into the contiguous memory */
14    for (int i = 0; i < rows; i++)
15        matrix[i] = &(temp_matrix[i * columns]);
16
17    free(temp_matrix);
18
19    return matrix;
20 }
```

In this way the memory is contiguous and we can use the matrix as a 2D array.